

Investigating Expert Collapse in EveNet MoE

Overview

- Simple dataset (MNIST)
- Baseline (non-MoE) transformer model
- Simple MoE implementation
- Isolated EveNet MoE implementation

Baseline Transformer (ViT)

- Single-layer Vision Transformer on 28×28 images, patched to 14×14
- CLS token + learnable positional embeddings
- Self-attention $\rightarrow$ LayerNorm $\rightarrow$ **standard FFN MLP** $\rightarrow$ LayerNorm $\rightarrow$ linear classifier
- FFN: two Linear layers (hidden dim 128, ReLU) - every token passes through the same network

MoE Recap

- Core idea: dynamically routes inputs to specialized subnetworks ("experts")
- Each expert is a subnetwork (the number of experts is a hyperparameter)
 - In a transformer architecture they replace the feedforward layer
- Purpose is to allow specialization on different input features
- A gating network determines which experts to activate for each input
- Only a subset of the experts are activated for each input (hyperparameter)
- Typically implemented as a classifier

Simple MoE Transformer

- Same backbone - FFN replaced with an 8-expert Mixture-of-Experts layer
- Top-2 hard: linear gate scores tokens, dispatches to 2 highest-scoring experts
- Tokens processed independently by their selected experts, recombined with softmax weights
- Simple load balancing term to encourage equal distribution of tokens

EveNet MoE

- 8 routed experts + 0 shared experts - same count as simple MoE for a fair comparison
- Top-2 routing with stochastic noise: extra learned noise head keeps exploration alive during training
- Two balancing terms: auxiliary loss ($\alpha = 0.01$) + logit-magnitude regularisation ($c_z = 0.001$)

Training Setup

Hyperparameter	Value
Dataset	MNIST (60K train / 10K test)
Embedding dim	64
Epochs	25
Batch size	64
Optimizer	Adam, LR 0.001 + cosine annealing
Device	CUDA

Both MoE variants trained from scratch, independently of each other and the baseline.

Performance Comparison

Metric	Baseline	Simple MoE	EveNet MoE
Test Accuracy	96.50 %	92.88 % (−3.6 pp)	96.61 % (+0.1 pp)
Test Loss	0.119	0.247 (2×)	0.150 (1.2×)
Final Train Loss	0.038	0.226	0.024

- Simple MoE underperforms - EveNet MoE matches baseline (wouldn't expect it to perform better in this case, but still interesting that specialisation doesn't hinder performance)

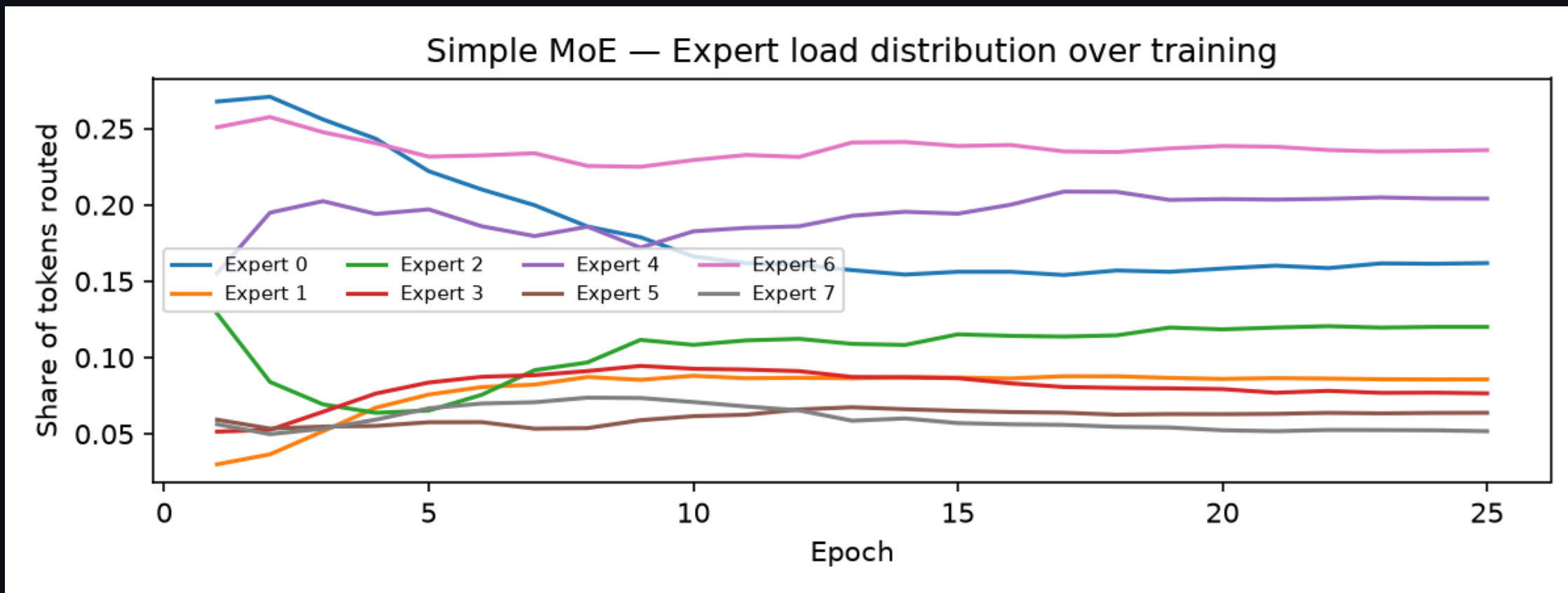
Expert Routing Distribution (Final Epoch) - Simple MoE

Expert	Share	Deviation
E0	16.2 %	+3.7 pp
E1	8.6 %	−3.9 pp
E2	12.0 %	−0.5 pp
E3	7.7 %	−4.8 pp
E4	20.4 %	+7.9 pp
E5	6.4 %	−6.1 pp
E6	23.6 %	+11.1 pp
E7	5.2 %	−7.3 pp

Expert Routing Distribution (Final Epoch) - EveNet MoE

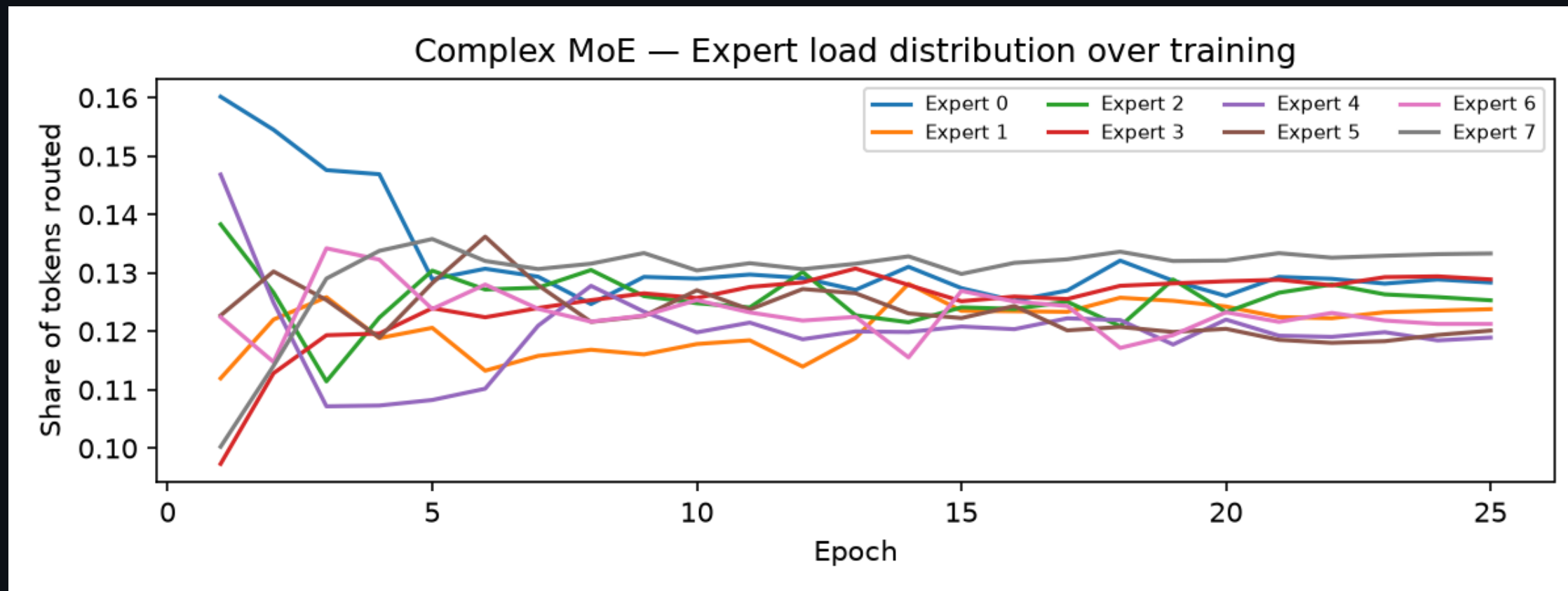
Expert	Share	Deviation
E0	12.4 %	−0.1 pp
E1–E6	12.5–12.7 %	≤ +0.2 pp
E7	11.8 %	−0.7 pp

Simple MoE - Routing Evolution



- Expert 6 absorbs ~24 % - nearly double its fair share (12.5 %)

EveNet MoE - Routing Evolution



- Nearly uniform from epoch 4 onward - all experts within ± 1.5 pp of fair share

Key Takeaways

- Simple MoE shows severe expert collapse
- EveNet MoE achieves near-uniform routing within 4 epochs and maintains it
 - Shows that the standard routing-collapse fixes implemented do actually work

Three Main Areas of Investigation

These discoveries highlight 3 main areas for continued investigation:

1. EveNet Training Regime
2. EveNet Architecture
3. HEP Data Itself

Area 1: EveNet Training Regime

- As this is a smaller, isolated testing model, it inherently is not a foundation model, where the concept of generalisation across tasks does not exist and is not being replicated
- Auxillary losses being overshadowed by task-specific losses
 - Could be tested by ablating over MoE loss term values to look for any changes in expert utilisation distribution

Area 2: EveNet Architecture

- The existing issues with scaling the model up could potentially be impacting the expert specialisation
 - So when the next version of the model is released, performance of MoE may be different (especially if we can test the bigger variants as well)
- Stacking transformer blocks is may propagate early expert bias (unlikely the *cause* though)

Area 3: HEP Data Itself

- The vision tokens used in this isolated simplified task are fundamentally different to pointcloud HEP data
- Need to better understand the distribution of the 500 million monte carlo data points that pretraining happens on
 - Could it be that these are skewed towards in-distribution standard model, where specialisation is less useful than on the downstreams?
- Does SIC lend itself towards a smooth representation in a way that discrete experts are harder to assign?

Next Steps

- **Best case: Ablation of hyperparameters to look for patterns**
- Explore the previously mentioned 3 areas of investigation
- Implement a framework for investigating what tokens (and more broadly 'type' of tokens) are being routed to each expert
- Get the smaller EveNet model working (potentially the place to start ablating)
- Explore expert voting as a fix in parallel to identifying the actual cause